PDF ② 전체 핵심 요점정리 (Core Summary & Full Evaluation)

대상 범위: SSAFY DB — SQL 01 (153p) · SQL 02 (113p) · Many To One Relationships 01 (81p) · Many To One Relationships 02 (76p)

설계 목적: 개념 간 인과 관계·성립 조건·실무 적용 사례(5대 요건 충족)·결정적 비교표·원본 62문항 전수 해설 및 7유형 심화 평가 통합본

0. 전체 학습 아키텍처 및 개념 간 선행·의존 관계도

원본 4권의 교육과정은 독립된 조각이 아니라 **① 왜 데이터를 테이블로 나누어 저장하는가(RDB & 정규화) → ② 단일 테이블에서 데이터를 어떻게 조건부로 추출·집계하는가(DQL) → ③ 테이블을 직접 만들고 데이터를 조작하며 나뉜 테이블을 어떻게 다시 합쳐 조회하는가(DDL·DML·JOIN) → ④ 이 관계형 원리를 Python Django 웹 애플리케이션의 댓글과 회원 권한 시스템에 어떻게 구현하는가(Many-to-One ORM & ERD)**라는 하나의 유기적인 흐름으로 연결되어 있다.

단원 구분	원본 매핑 범위	핵심 학습 목표 및 선행 의존성	결과를 바꾸는 핵심 조건·예외
Chapter 1 DB·RDB·SQL 기초	db 1.pdf p.1~45 (C01 ~ C04)	파일/스프레드시트 한계 → RDB 구조(PK/FK) → DBMS/SQLite → SQL 4대 분류(DDL·DQL·DML·DCL)	동명이인 식별을 위한 PK(고유·NOT NULL)와 테이블 연결용 FK(중복 가능) 구분
Chapter 2 단일 테이블 DQL	db 1.pdf p.46~152 (C05 ~ C10)	SELECT → ORDER BY → DISTINCT → WHERE(연산자·NULL) → LIMIT → GROUP BY / HAVING	정수 나눗셈 버림, SQLite NULL 오름차순 최상단, 3값 논리(IS NULL), 7단계 실행 순서
Chapter 3 DDL·DML·JOIN	db 2.pdf p.1~112 (C11 ~ C14)	CREATE/ALTER/DROP TABLE, 타입 선호도 → INSERT/UPDATE/DELETE(서브쿼리) → INNER/LEFT JOIN	INTEGER PRIMARY KEY AUTOINCREMENT 강제, ADD COLUMN NOT NULL DEFAULT, LEFT JOIN + IS NULL
Chapter 4 Django N:1 (댓글)	db 3.pdf p.1~80 (C15 ~ C17)	Comment(N):Article(1) FK 위치 → ForeignKey & Migration → 참조 vs 역참조 → 댓글 CRUD(commit=False)	FK는 N쪽 배치, article_id 자동 생성, comment_set 역참조 매니저, 폼 FK 제외 후 주입
Chapter 5 User N:1·데코레이터·ERD	db 4.pdf p.1~75 (C18 ~ C20)	Article/Comment & User(AUTH_USER_MODEL) → 작성자 권한 제어 → View Decorators(405) → ERD(Crow's foot)	지연 평가(문자열 vs 클래스), 기존 행 마이그레이션 기본값, 템플릿+뷰 이중 방어, @require_safe

Chapter 1. 데이터베이스와 관계형 데이터베이스(RDB), SQL 문법 체계 (db 1.pdf 전반부)

1.1 왜 파일이나 스프레드시트 대신 데이터베이스(DB)를 사용하는가? [C01]

- **정확한 의미와 필요성**: **데이터(Data)**는 컴퓨터가 처리할 수 있도록 구조화한 정보이며, **데이터베이스(Database)**는 체계적으로 정리된 데이터의 모음이다. 일반 텍스트 파일(개인정보.txt)은 구조적 검색이 불가능하고, 스프레드시트(엑셀)는 행과 열을 제공하지만 실무 서비스에 적용할 때 다음 **3가지 치명적 한계**가 발생한다:
  1. **크기(Size)의 한계**: 일반적으로 약 **100만 행까지만** 저장 가능하여 연간 수억 건의 주문 데이터를 감당할 수 없다.

2. **보안(Security)의 한계**: 단순히 파일이나 링크 소유 여부에 따른 접근 권한만 제공할 뿐, 테이블·열·행 단위의 정밀한 권한 제어가 불가능하다.
3. **정확성(Accuracy)의 한계**: 만약 공식 지명인 "강원"이 "강언"으로 바뀌었을 때, 데이터가 여러 시트에 중복 분산되어 있으면 모든 위치를 찾아 일일이 업데이트해야 하며 필연적으로 **변경 누락이나 데이터 불일치**가 발생한다.

- **데이터베이스의 핵심 역할 (CRUD)**: 데이터를 구조적으로 저장하고 **Create(생성), Read(조회), Update(수정), Delete(삭제)**의 4가지 기본 조작을 일관성 있게 수행한다.

1.2 관계형 데이터베이스(RDB)의 구조와 기본 키(PK) · 외래 키(FK) [C02, C03]

- **왜 관계(Relationship)가 필요한가?**: 고객 정보와 주문 내역을 하나의 표에 합쳐 적으면, 한 고객이 10번 주문할 때마다 고객의 이름·청구지·주소지가 10번 중복 저장된다. 반대로 고객 표와 주문 표를 쪼개 놓되 연결고리가 없으면, 동명이인('김한웅'이 2명)이 있을 때 누가 주문했는지 구별할 수 없다.
- **해결 원리 (PK와 FK에 의한 논리적 연결)**:
  - **기본 키(Primary Key, PK)**: 각 테이블에서 개별 레코드(행)를 유일하게 식별하는 고유 번호(예: 주민등록번호, 고객 테이블의 `id`). **절대 중복될 수 없으며 `NULL` 값을 가질 수 없다.**
  - **외래 키(Foreign Key, FK)**: 다른 테이블의 기본 키(PK)를 참조하는 필드(예: 주문 테이블의 고객 `ID`). 데이터를 각각의 테이블에 나누어 저장하되 공통된 키 값(PK ↔ FK)을 통해 논리적 **관계(Relationship)**를 맺어 필요할 때 `JOIN`으로 함께 조회한다.
- **실제 적용 사례 (원본 `db 1.pdf` p.19~31 고객·주문 테이블 완벽 재현)**:
  - *① 구체적 상황*: 쇼핑몰에서 1번 김한웅(청구지: 서울, 주소지: 강원), 2번 유미선(강원, 경기), 3번 박지수(경기, 서울) 세 고객의 주문을 관리한다. 박지수가 `2002/01/01`(배송완료)과 `2002/03/04`(상품 준비중) 두 번 주문하고, 김한웅이 `2002/02/03`(배송중) 한 번 주문했다.
  - *② 왜 필요한가*: 박지수가 이사하여 주소지가 바뀌었을 때, 주문 테이블마다 주소를 고치지 않고 고객 테이블의 3번 행만 수정해도 모든 주문에 최신 주소가 반영되어야 한다.
  - *③ 실제 적용 행동*: 주문 테이블에는 고객 이름이나 주소를 적지 않고 고객 `ID`(외래 키) 컬럼만 두어 `3, 3, 1`을 저장한다.
  - *④ 결과 확인*: 1번 주문과 2번 주문의 고객 `ID`가 모두 `3`이므로 고객 테이블의 `id = 3`(박지수)과 정확히 연결된다.
  - *⑤ 잘못 적용했을 때의 차이*: 만약 외래 키 없이 고객 이름(`'박지수'`)으로 연결했다면, 동명이인인 박지수가 가입하는 순간 기존 주문이 누구의 것인지 판별 불가능해진다.

1.3 SQL 문법 기본 규칙과 4대 Statement 유형 비교 [C04]

- **SQL Syntax 2대 기본 규칙 (`db 1.pdf` p.42)**:
  1. SQL 키워드(`SELECT, FROM, WHERE` 등)는 **대소문자를 구분하지 않는다**. 단, 테이블명·컬럼명과 명시적으로 구분하기 위해 **대문자로 작성하는 것을 권장**한다.
  2. 각 SQL Statements의 끝에는 명령어의 마침표 역할을 하는 **세미콜론(;)이 반드시 필요**하다.
- **SQL Statement vs Clause vs Keyword 구분**: 예를 들어 `SELECT column_name FROM table_name;` 전체 코드는 하나의 **SELECT Statement(문장)**라 부르며, 이 문장은 `SELECT column_name`과 `FROM table_name`이라는 2개의 **Clause(절)**, 그리고 `SELECT`와 `FROM`이라는 2개의 **Keyword(키워드)**로 구성된다.

Chapter 2. 단일 테이블 데이터 조회(DQL)와 조건·정렬·집계 파이프라인 (db 1.pdf 후반부)

2.1 SELECT 조회와 산술 연산 시 정수 나눗셈 함정 [C05]

- **핵심 원리 및 조건**: SELECT 절에서는 단순 필드명뿐 아니라 +, -, *, / 같은 산술 연산식을 사용할 수 있고, AS '별칭'으로 결과 컬럼의 표시 이름을 변경할 수 있다.
- **실제 적용 사례 (tracks 테이블 밀리초 → 분 단위 변환, db 1.pdf p.61~62)**:
  - *① 상황 및 필요성*: 음원 테이블 tracks에는 곡 길이가 밀리초(Milliseconds, 정수형)로 저장되어 있어 사용자가 몇 분 짜리 곡인지 직관적으로 알기 어렵다. 이를 60,000으로 나누어 분 단위로 출력해야 한다.
  - *② 적용 행동 및 잘못 적용했을 때의 차이*:

```
-- [적용 1: 정수 나눗셈] 소수점 이하가 버려짐!
SELECT Name, Milliseconds / 60000 AS '재생 시간(분)' FROM tracks;
-- 결과: 343719ms (약 5.73분) -> 5 출력, 230619ms (약 3.84분) -> 3 출력

-- [적용 2: 실수 나눗셈] 소수점까지 정확히 유지하려면 60000.0으로 나눔!
SELECT Name, Milliseconds / 60000.0 AS '재생 시간(분)' FROM tracks;
```

2.2 ORDER BY 정렬 기준과 SQLite의 NULL 정렬 위치 [C06]

- **다중 컬럼 정렬의 작동 원리 (db 1.pdf p.71~72)**: ORDER BY Country DESC, City;처럼 여러 컬럼을 지정하면, **1차 기준(Country DESC)으로 먼저 전체를 정렬**한 뒤, **Country 값이 동일한 행들 내부에서만 2차 기준(City ASC, 기본값)으로 정렬**한다. 주의할 점은 정렬 방식(ASC/DESC)이 컬럼마다 개별 적용되므로 Country DESC, City에서 City는 내림차순이 아니라 **오름차순(ASC)**이다!
- **정렬에서의 NULL 취급 (db 1.pdf p.75)**: employees 테이블에서 직속 상사 사번(ReportsTo)을 오름차순 정렬(ORDER BY ReportsTo;)하면, 상사가 없는 사장(Andrew Adams)의 (NULL)이 **가장 먼저(맨 첫 줄에)** 출력된다. **SQLite는 정렬 시 NULL을 가장 작은 값으로 취급**하기 때문이다(따라서 DESC 내림차순 시에는 맨 마지막에 출력됨).

2.3 DISTINCT와 WHERE 필터링 연산자 (BETWEEN, IN, LIKE, IS NULL) [C07, C08]

- **DISTINCT 사용 조건 (db 1.pdf p.80~84)**: 중복된 값을 제거하고 고유한 결과만 조회할 때 사용하며, 반드시 **SELECT 키워드 바로 뒤에** 작성해야 한다(SELECT DISTINCT Country FROM customers;).
- **주요 필터링 연산자 선택 기준 및 주의점**:
  - **범위 비교 (BETWEEN A AND B)**: Bytes BETWEEN 100000 AND 500000은 100000 이상 500000 이하(경계값 포함)를 조회한다. 만약 이 결과를 Bytes 오름차순으로 정렬하고 싶다면 ORDER BY Bytes는 반드시 **WHERE 절 뒤에** 작성해야 한다(db 1.pdf p.98).
  - **목록 포함/제외 (IN / NOT IN)**: Country IN ('Canada', 'Germany', 'France')는 OR 연산자 여러 개를 묶은 것과 같으며, 여집합은 NOT IN으로 조회한다.
  - **패턴 매칭 (LIKE + % / _)**: 단순 동등 비교(=)가 아니라 부분 문자열이나 글자 수를 검사할 때 반드시 LIKE를 쓴다. 예: FirstName LIKE '___a'는 앞의 3글자(___) + 마지막 a이므로 **정확히 4글자이면서 'a'로 끝나는 이름**(Kara, Emma, Puja)만 추출한다.
- **결정적 비교표 1: 일반 동등 연산자(=) vs IS 연산자와 SQL 3값 논리 (db 1.pdf p.136~138)**

비교 항목	= (등호 연산자)	IS / IS NOT 연산자
사용 목적	일반적인 실제 값(숫자, 문자열)의 동등성 비교	NULL(값의 부재) 또는 Boolean(TRUE/FALSE) 비교
NULL 비교 시 동작 (SQL 3값 논리)	피연산자 중 하나라도 NULL이면 결과는 TRUE도 FALSE도 아닌 제3의 값 **UNKNOWN(알 수 없음)**이 됨 (심지어 NULL = NULL도 UNKNOWN!)	해당 컬럼에 값이 비어 있는지 여부를 명확히 판단하여 **TRUE 또는 FALSE**를 반환함

비교 항목	= (등호 연산자)	IS / IS NOT 연산자
실패 상황 vs 올바른 쿼리	WHERE Company = NULL 실행 시 조건이 TRUE인 행이 하나도 없어 **0건 반환(조회 실패)**	WHERE Company IS NULL / WHERE is_active IS TRUE

2.4 LIMIT ... OFFSET과 GROUP BY / HAVING 및 7단계 실행 순서 [C09, C10]

- **LIMIT과 OFFSET 구간 계산법 (db 1.pdf p.111~116)**: LIMIT row_count OFFSET offset;에서 offset은 0부터 시작하여 **앞에서 몇 개를 건너뛸지**를 뜻한다. 따라서 **"N번째부터 M번째까지"**를 조회하라는 문제가 나오면 공식처럼 다음 두 값을 대입한다:
  - 건너뛸 개수: **OFFSET = N - 1** (예: 4번째부터면 앞의 3개를 건너뛰므로 OFFSET 3)
  - 가져올 개수: **LIMIT = M - N + 1** (예: 4번째~7번째면 4, 5, 6, 7 총 4개이므로 LIMIT 4)
- **결정적 비교표 2: WHERE 절 vs HAVING 절 (db 1.pdf p.126~130)**

비교 기준	WHERE 절	HAVING 절
필터링 대상	그룹화하기 전의 **개별 행(Row)**	GROUP BY에 의해 묶인 **그룹(Group) 및 집계 결과**
적용 시점 (실행 순서)	FROM(및 JOIN) 직후, **GROUP BY 이전에** 평가 (2단계)	GROUP BY 및 집계 함수 계산 **이후에** 평가 (4단계)
집계 함수 사용 가능 여부	**불가능!** WHERE AVG(Milliseconds/60000) < 10 실행 시 **misuse of aggregate: AVG() 에러** 발생	**가능!** GROUP BY department HAVING COUNT(*) > 5;
실제 선택 기준	"급여가 30,000 초과인 직원만"처럼 원본 컬럼 값 자체를 거를 때 선택 (WHERE salary > 30000)	"부서원 수가 5명 초과인 부서만", "평균 재생 시간이 10분 미만인 작곡가만"처럼 집계값을 거를 때 선택

- **SELECT statement 7단계 실행 순서와 쿼리 해석 (db 1.pdf p.131, 151)**:
  - **문법 작성 순서**: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
  - **내부 엔진 실행 순서**: ① **FROM** (대상 테이블 로드) → ② **WHERE** (조건에 맞는 개별 행 필터링) → ③ **GROUP BY** (지정 컬럼 기준 그룹화) → ④ **HAVING** (그룹별 집계 조건 필터링) → ⑤ **SELECT** (출력할 컬럼 선택 및 AS 별칭 생성) → ⑥ **ORDER BY** (결과 정렬 — 5단계에서 만든 별칭 사용 가능!) → ⑦ **LIMIT** (최종 출력 행 개수 제한).

Chapter 3. 테이블 스키마 관리(DDL) · 데이터 조작(DML) · 다중 테이블 JOIN (db 2.pdf)

3.1 테이블 생성(CREATE TABLE), 제약 조건, SQLite 타입 선호도 [C11]

- **테이블 생성과 스키마 검증 (db 2.pdf p.10~15)**:

```
CREATE TABLE examples (
    ExamId INTEGER PRIMARY KEY AUTOINCREMENT,
    LastName VARCHAR(50) NOT NULL,
    FirstName VARCHAR(50) NOT NULL
);
PRAGMA table_info('examples');
```

  PRAGMA table_info('examples');를 실행하면 각 컬럼의 cid(0부터 시작하는 컬럼 ID), name, type, notnull(NOT NULL이면 1, 아니면 0), dflt_value(기본값, 미지정 시 내부적으로 NULL), pk(기본키면 1)가 출력된다.
- **AUTOINCREMENT의 3가지 핵심 규칙 (db 2.pdf p.19)**:
  1. 새 레코드에 대해 항상 **이전에 존재했던 최대값보다 큰 고유 정수값**을 자동 할당한다.
  2. 한 번 **삭제된 행의 PK 값은 절대 재사용되지 않는다**.

3. SQLite에서는 반드시 **INTEGER PRIMARY KEY AUTOINCREMENT**로만 선언해야 한다(INT나 BIGINT로 쓰면 문법 에러 발생).

- **SQLite 타입 선호도(Type Affinity)와 날짜/시간 (db 2.pdf p.16, 94~100)**: SQLite는 타 DBMS와의 호환성 및 유연한 데이터 처리를 위해 5가지 기본 스토리지 클래스(NULL, INTEGER, REAL, TEXT, BLOB)와 **타입 선호도(Type Affinity)**를 사용한다.
  - 컬럼 타입에 INT, TINYINT, BIGINT 등 "INT"가 포함되면 → **INTEGER 선호도**
  - CHARACTER(20), VARCHAR(255), NVARCHAR(100), TEXT, CLOB 등 문자 관련 키워드가 포함되면 → **TEXT 선호도** (단, VARCHAR(50)의 길이 50은 검사하지 않음!)
  - BLOB 또는 타입을 아예 지정하지 않으면 → **BLOB 선호도**
  - REAL, DOUBLE, FLOAT가 포함되면 → **REAL 선호도**
  - 그 외 NUMERIC, DECIMAL, BOOLEAN, DATE, DATETIME 등은 → **NUMERIC 선호도**
  - **날짜와 시간**: 별도의 날짜 전용 타입이 없으므로 DATE() 같은 함수를 통해 표기 형식에 따라 TEXT(예: '2025-07-29'), REAL, INTEGER 값으로 저장한다.

3.2 ALTER TABLE 스키마 변경 4대 명령과 DROP TABLE [C12]

- **4가지 ALTER TABLE 역할 및 문법 (db 2.pdf p.22~36)**:
  - **필드 추가 (ADD COLUMN)**: ALTER TABLE examples ADD COLUMN Country VARCHAR(100) NOT NULL DEFAULT 'default value';
    핵심 제약: ① 추가하려는 필드에 **NOT NULL 제약조건이 있을 경우 반드시 NULL이 아닌 기본값(DEFAULT ...)을 함께 설정**해야 한다. ② SQLite는 단일 ALTER TABLE 문으로 한 번에 여러 필드를 추가할 수 없다.
  - **필드 이름 변경 (RENAME COLUMN)**: ALTER TABLE examples RENAME COLUMN Address TO PostCode;
  - **필드 삭제 (DROP COLUMN)**: ALTER TABLE examples DROP COLUMN PostCode;
  - **테이블 이름 변경 (RENAME TO)**: ALTER TABLE examples RENAME TO new_examples;
- **테이블 삭제 (DROP TABLE)**: DROP TABLE new_examples;는 테이블의 구조와 내부 데이터를 통째로 영구 삭제한다(ALTER TABLE로는 테이블 전체 삭제 불가).

3.3 데이터 조작어(DML: INSERT, UPDATE, DELETE)와 서브쿼리 활용 [C13]

- **INSERT INTO 3가지 활용 패턴 (db 2.pdf p.46~52)**:

```
-- 1) 단일 행 삽입 (id는 AUTOINCREMENT이므로 생략 가능)
INSERT INTO articles (title, content, createdAt) VALUES ('hello', 'world', '2000-01-01');
-- 2) 다중 행 동시 삽입 (VALUES 뒤에 콤마로 튜플 나열)
INSERT INTO articles (title, content, createdAt)
VALUES ('title1', 'content1', '1900-01-01'), ('title2', 'content2', '1800-01-01'), ('title3', 'content3',
'1700-01-01');
-- 3) SQLite 내장 DATE() 함수로 오늘 날짜 자동 입력
INSERT INTO articles (title, content, createdAt) VALUES ('mytitle', 'mycontent', DATE());
```

- **UPDATE와 DELETE의 필수 조건 (db 2.pdf p.55~67)**:
  - UPDATE articles SET title = 'update Title', content = 'update Content' WHERE id = 2;
  - DELETE FROM articles WHERE id = 1;
  - *치명적 주의사항*: UPDATE와 DELETE에서 **WHERE 절을 생략하면 에러가 나는 것이 아니라 테이블의 모든 레코드가 수정되거나 삭제**된다!
  - *실제 적용 사례 (서브쿼리를 이용한 "가장 오래된 게시글 2개 삭제", db 2.pdf p.65~67)*: 표준 SQLite DELETE 문에서는 직접 ORDER BY와 LIMIT을 쓰기보다, 먼저 SELECT id FROM articles ORDER BY createdAt LIMIT 2 서브쿼리로 삭제할 대상 ID(4번, 3번)를 추출한 뒤 바깥 쿼리의 WHERE id IN (...)에 전달하여 안전하게 삭제한다.

3.4 다중 테이블 쿼리(JOIN)와 INNER JOIN vs LEFT JOIN [C14]

테이블 분리의 이유와 JOIN이 필요한 순간 (db 2.pdf p.70~74): 하나의 게시판 테이블에 id, title, content, writer, role을 모두 넣으면 ① 동명이인 작성자를 구별할 수 없고, ② 작성자 이름('하석주' → '권미숙')이나 역할이 바뀔 때 그 사람이 쓴 모든 게시글 행을 찾아 수정해야 한다. 따라서 articles(게시글), users(회원), roles(역할)로 **테이블을 나누어 외래 키(userId,**

roleId)로 연결한다. 이렇게 나누면 관리·수정은 쉬워지지만, 화면에 게시글과 작성자 이름을 함께 출력하려면 나뉜 테이블을 다시 결합하는 **JOIN**이 필요하다.

- **실제 적용 사례 (커뮤니티 게시판 3-테이블 외래 키 분리 설계, db 2.pdf p.72~73)**: articles 테이블에는 userId 외래 키를 두어 누가 작성했는지 연결하고, users 테이블에는 roleId 외래 키를 두어 roles 테이블(1: admin, 2: staff, 3: student)과 연결한다. 이렇게 분리하면 ① 관리자만 보고 싶을 때 roleId = 1만 조회하면 되고, ② 1번 회원 하석주가 권미숙으로 개명하더라도 users 테이블에서 단 한 번만 이름을 변경하면 그 사람이 작성한 모든 게시글에 자동으로 새 이름이 반영된다.
- **결정적 비교표 3: INNER JOIN vs LEFT JOIN (db 2.pdf p.78~92 실습 완벽 대조)**
  전제 데이터: users 테이블에는 (1: 하석주), (2: 송윤미), (3: 유하선) 3명이 있고, articles 테이블에는 1번글(userId=1), 2번글(userId=2), 3번글(userId=1), 4번글(userId=4, 존재하지 않는 회원!), 5번글(userId=1) 총 5건이 저장되어 있다(SQLite는 기본적으로 외래 키 검사를 하지 않아 userId=4도 저장됨).

비교 기준	INNER JOIN (내부 조인)	LEFT JOIN (왼쪽 외부 조인)
조인 원리	두 테이블에서 ON 조건 값이 **양쪽 모두 일치하는 행만** 반환 (교집합)	FROM 절의 **왼쪽 테이블 행은 무조건 모두 유지**하고, 오른쪽에 일치하는 행이 없으면 **NULL**로 채움
실행 쿼리	SELECT * FROM articles INNER JOIN users ON users.id = articles.userId;	SELECT * FROM articles LEFT JOIN users ON users.id = articles.userId;
4번 게시글 (userId=4) 처리	users 테이블에 id=4가 없으므로 **결과에서 완전히 제외됨** (총 4행 반환: 1, 2, 3, 5번 글)	왼쪽(articles) 행이므로 **삭제되지 않고 살아남으며**, 오른쪽 users.id와 users.name 칸에 **NULL**이 표시됨 (총 5행 반환)
특수 활용 패턴	**작성자가 실제로 존재하는** 유효한 게시글만 작성자 이름과 함께 출력할 때 (WHERE users.id = 1 결합 가능)	**LEFT JOIN + WHERE IS NULL ("~한 적 없는" 찾기)**: SELECT users.name FROM users LEFT JOIN articles ON articles.userId = users.id WHERE articles.userId IS NULL; → 글을 한 번도 안 쓴 **'유하선'** 검출!

Chapter 4. Django Many-to-One Relationships 01 — 댓글 모델과 역참조 (db 3.pdf)

4.1 Comment(N) : Article(1) 관계 설계와 ForeignKey / Migration [C15]

- **관계 판단 기준 (db 3.pdf p.9~15)**: 하나의 게시글(1)에는 여러 개의 댓글(N)이 달릴 수 있고(0개일 수도 있음), 하나의 댓글은 여러 게시글에 동시에 달릴 수 없다. 따라서 **Comment(N) : Article(1)의 다대일(Many-to-One) 관계**다. (반면 영화 Movie와 배우 Actor는 한 영화에 여러 배우가 출연하고 한 배우도 여러 영화에 출연하므로 N:1이 아니라 **N:M 관계**다!).
- **외래 키(ForeignKey) 배치와 모델 정의 (db 3.pdf p.14~21)**: 외래 키는 반드시 **N을 담당하는 Comment 클래스**에 작성한다.

```
# articles/models.py
class Comment(models.Model):
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    content = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

  - **필수 인자 2개**: 참조할 모델 클래스(to = Article)와 부모 삭제 시 동작(on_delete = models.CASCADE 등).
  - **단수형 변수명과 DB 컬럼명 규칙**: Comment 인스턴스 하나는 단일 Article 객체를 가리키므로 변수명을 소문자 단수형 **article**로 짓는다. Migration을 수행하면 DB의 articles_comment 테이블 마지막 열에 **article_id**(자료형: bigint) 필드가 자동 생성된다.

4.2 Django Shell 댓글 생성과 참조(Reference) vs 역참조(Reverse Reference) [C16]

- **왜 외래 키 없이 comment.save()를 하면 에러가 나는가? (db 3.pdf p.24~25)**: comment = Comment(content='first comment') 후 바로 comment.save()를 실행하면 **IntegrityError: NOT NULL constraint failed: articles_comment.article_id**가 발생한다. 어떤 게시글에 달린 댓글인지 외래 키 정보가 누락되었기 때문이다. 이때 comment.article_id = article.pk처럼 정수 PK를 직접 넣는 것보다 **comment.article = article처럼 모델 인스턴스 객체를 직접 할당**하는 것이 타입 검증 측면에서 안전하여 권장된다.
- **결정적 비교표 4: 정방향 참조(Reference) vs 역참조(Reverse Reference) (db 3.pdf p.26~40)**

비교 기준	참조 (Reference, N → 1)	역참조 (Reverse Reference, 1 → N)
출발 방향 및 조건	ForeignKey 필드가 **정의되어 있는 모델(N, Comment)**에서 부모 모델(1, Article)을 조회	ForeignKey 필드가 **없는 모델(1, Article)**에서 자신을 참조하는 자식 모델(N, Comment)들을 거꾸로 조회
코드 문법	comment.article (객체 반환) comment.article.pk / comment.article_id (PK 값) comment.article.content (부모 게시글 내용)	**article.comment_set.all()** (구조: 모델인스턴스 . related_manager . QuerySet_API)
내부 DB 동작	내 행(articles_comment)에 저장된 article_id 칸의 값을 읽어 해당 게시글을 가져옴	articles_comment 테이블에서 article_id = 내 pk인 행들을 모두 찾아 QuerySet 리스트로 반환함
주의할 오개념	comment.article_pk라는 속성은 존재하지 않음! (article_id 또는 article.pk만 가능)	related_name을 별도 지정하지 않으면 기본 매니저 이름은 **소문자모델명_set(comment_set)**으로 자동 생성됨

4.3 웹 애플리케이션 댓글 CREATE · READ · DELETE 구현과 템플릿 UX [C17]

- **댓글 CREATE 12단계 핵심 원리 (db 3.pdf p.43~54)**:
  1. **CommentForm 외래 키 출력 제외**: fields = '__all__'로 두면 사용자가 화면에서 게시글(Article)을 직접 선택하는 드롭다운이 나타나 댓글 작성 불가 글에 댓글을 달거나 엉뚱한 글에 댓글을 다는 문제가 생긴다. 따라서 **fields = ('content',)**(튜플 마지막 콤마 필수!)로 수정하여 본문만 입력받는다.
  2. **단일 책임 원칙(SRP)과 Variable Routing**: 댓글 저장은 게시글 상세 조회(detail 함수, GET)와 전혀 다른 기능이므로 별도의 **comments_create(request, pk)** 뷰 함수(POST 전용)로 분리한다. 이때 폼에서 제외된 게시글 PK는 URL의 **Variable Routing(path('<int:pk>/comments/', views.comments_create, name='comments_create'))**을 통해 전달받는다.
  3. **save(commit=False)를 통한 외래 키 주입**:

```
def comments_create(request, pk):
    article = Article.objects.get(pk=pk)
    comment_form = CommentForm(request.POST)
    if comment_form.is_valid():
        comment = comment_form.save(commit=False) # DB 저장 보류, 인스턴스만 생성!
        comment.article = article                 # URL로 조회한 게시글 객체 주입
        comment.save()                            # 최종 DB INSERT 수행
        return redirect('articles:detail', article.pk)
    context = {'article': article, 'comment_form': comment_form}
    return render(request, 'articles/detail.html', context)
```

- **댓글 READ와 DELETE 및 추가 기능 (db 3.pdf p.55~70)**:
  - **READ**: detail(request, pk) 뷰에서 comments = article.comment_set.all()로 해당 글의 댓글만 역참조하여 context로 넘긴다(절대 Comment.objects.all()을 쓰면 안 됨!).

- **DELETE**: URL을 '<int:article_pk>/comments/<int:comment_pk>/delete/'로 설계한다. 삭제 자체는 comment_pk만 있으면 되지만, **삭제 완료 후 다시 원래 게시글 상세 페이지(redirect('articles:detail', article_pk))로 돌아가기 위해 article_pk도 함께 전달**받는다.
- **DTL for...empty와 개수 출력**: 댓글이 하나도 없을 때 별도의 {% if %} 없이 {% for comment in comments %} ... {% empty %} 댓글이 없어요.. {% endfor %}로 대체 문구를 출력하며, 댓글 개수는 {{ comments|length }} 또는 {{ article.comment_set.count }}로 표시한다.

Chapter 5. User N:1 관계 · 작성자 권한 제어 · View Decorators · ERD (db 4.pdf)

5.1 Article & User, Comment & User 관계 설정과 Migration [C18]

- **왜 models.py에서는 User 클래스를 직접 import하지 않고 settings.AUTH_USER_MODEL을 쓰는가? (db 4.pdf p.8~9)**: Django 앱들이 로딩될 때 articles/models.py 생성 시점이 accounts 앱의 User 클래스 로딩 시점보다 빠르면, User 모델을 찾지 못해 에러가 발생할 수 있다. 따라서 models.py에서는 문자열('accounts.User')을 반환하는 **settings.AUTH_USER_MODEL**을 넣어 모든 모델이 완전히 로딩된 뒤 클래스를 연결하는 **지연 평가(Lazy Evaluation)** 방식을 사용하고, models.py를 제외한 나머지 모든 파일(views.py, forms.py 등)에서는 **get_user_model()** 함수를 사용한다.
- **기존 테이블에 NOT NULL 외래 키 추가 시 Migration 2단계 프롬프트 (db 4.pdf p.10~12, 30~31)**: 이미 게시글이나 댓글이 들어 있는 상태에서 user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)를 추가하고 python manage.py makemigrations를 실행하면, Django는 기존 행들의 user_id(기본적으로 NOT NULL)를 무엇으로 채울지 묻는다:
  - 1단계 프롬프트: 1) Provide a one-off default now → **1 입력 후 Enter**
  - 2단계 프롬프트: Please enter the default value as valid Python → **1 입력 후 Enter** (기존에 작성된 모든 게시글/댓글을 1번 회원이 작성한 것으로 처리함. 단, DB에 1번 회원이 없으면 migrate 시 에러 발생!).

5.2 게시글·댓글 작성자 저장과 수정·삭제 권한 이중 제어 [C18]

- **사칭 방지 및 작성자 자동 주입 (db 4.pdf p.14~18, 33~35)**: ArticleForm에 user 필드를 그대로 두면 글 작성 화면에 작성자 선택 드롭다운이 노출되어 타인을 사칭할 수 있다. 따라서 ArticleForm의 fields = ('title', 'content',)로 user를 빼고, 그냥 저장하면 발생하는 IntegrityError (NOT NULL constraint failed: articles_article.user_id)를 막기 위해 뷰 함수에서 save(commit=False) 후 현재 로그인한 사용자 객체인 **request.user**를 할당한다:

```
# 게시글 CREATE (articles/views.py)
article = form.save(commit=False)
article.user = request.user
article.save()
```

```
# 댓글 CREATE (articles/views.py)
comment = comment_form.save(commit=False)
comment.article = article
comment.user = request.user
comment.save()
```

- **왜 템플릿(HTML)에서 버튼을 숨겼는데도 views.py에서 또 request.user를 검사하는가? (db 4.pdf p.23~26, 39~40)**:
  - **템플릿 UI 제어**: {% if request.user == article.user %} (댓글은 comment.user)로 작성자 본인에게만 수정/삭제 버튼을 출력한다.
  - **뷰 함수 서버 제어**: if request.user == article.user:일 때만 article.delete()나 수정을 실행한다. **이유**: 악의적인 사용자가 브라우저 버튼을 누르는 대신 **Python requests 패키지나 Postman 같은 API 툴을 이용해 삭제 URL로 직접 POST 요청을 보낼 수 있기 때문**에 서버 내부(views.py)에서도 반드시 작성자 본인 여부를 재검증해야 한다!

5.3 View Decorators와 허용 HTTP 메서드 제한 (405 Method Not Allowed) [C19]

- **View Decorator의 역할과 종류 (db 4.pdf p.41~49, 65)**: 뷰 함수 위에 붙여서 뷰의 코드 흐름을 방해하지 않고 로그인 여부나 HTTP 요청 방식을 사전 체크리스트처럼 제한하는 데코레이터다(from django.views.decorators.http import ...).
  - **@require_http_methods(['GET', 'POST'])**: 인자로 전달된 대문자 문자열 리스트의 HTTP method만 허용.
  - **@require_safe**: 조회 전용인 **GET과 HEAD method만 허용**. (@require_GET도 존재하지만, 표준 웹 클라이언트가 메타데이터 조회를 위해 보내는 HEAD 요청까지 포괄적으로 지원하는 **@require_safe 사용을 공식 권장**한다!)
  - **@require_POST**: 오직 **POST method만 허용** (예: delete, comments_create, comments_delete).

- **위반 시 반환 결과**: 허용되지 않은 HTTP method로 요청하면 **HttpResponseNotAllowed — 상태 코드 405 (Method Not Allowed) 에러**를 반환한다.

5.4 ERD (Entity-Relationship Diagram)와 Cardinality 표기법 [C20]

- **ERD 3대 구성 요소와 중요성 (db 4.pdf p.50~56, 60~62)**:
  - **엔티티(Entity, 개체)** = 데이터베이스 **테이블(Table)** (예: User, Article, Comment).
  - **속성(Attribute)** = 엔티티가 가지는 데이터 항목인 **컬럼(Column/Field)** 및 데이터 타입 (예: id (integer), name (varchar)).
  - **관계(Relationship)** = 엔티티 간의 연관성을 나타내는 **연결선(FK 참조선)**.
  - **중요성 및 도구**: 시스템 개발 전 중복 데이터를 제거하고 구조적 오류를 방지하며, 개발자·기획자·디자이너·비전문가 간 의사소통을 돕는 핵심 설계 도구다. 무료 제작 사이트로 **Draw.io (diagrams.net)**와 **ERDCloud**가 널리 쓰인다.
- **Cardinality(수적 관계)와 Crow's foot(까마귀 발) 적용 (db 4.pdf p.57~59)**: User와 Comment의 관계에서 "회원은 여러 댓글을 작성한다(0개일 수도 있음)"는 Comment 쪽 선 끝에 **Zero or many (—O<)** 기호를 그리고, "각 댓글은 반드시 하나의 작성자(회원)만 존재한다"는 User 쪽 선 끝에 **One and only one (—||—)** 기호를 그려 표현한다.

Chapter 6. 원본 교재 4권 확인 문제 전수 통합 해설 (총 62문항 100% 수록)

원본 4개 파일 말미에 수록된 확인 문제 **총 62문항(SQL 01 16문항 + SQL 02 16문항 + Many-to-One 01 14문항 + Many-to-One 02 16문항)**을 단 한 문제도 누락 없이 문제·정답·핵심 판단 근거와 함께 통합 정리하였다.

6.1 db 1.pdf (SQL 01) 확인 문제 1~16번 전수 해설 (원본 p.142~147)

번호	원본 문제 요지 및 빈칸/선지	정답	핵심 판단 근거 (해설)
1	모든 필드를 조회하는 SQL 문: SELECT _____ FROM employees;	* (SELECT * FROM employees;)	애스터리스크(*)는 테이블의 모든 필드(열)를 선택할 때 사용한다.
2	고객 테이블에서 이름과 국가 조회: SELECT _____, _____ FROM customers;	**FirstName, Country**	조회하려는 여러 컬럼 이름을 쉼표(,)로 구분하여 SELECT 뒤에 나열한다.
3	'국가'가 'USA'인 고객만 조회하는 WHERE 절: WHERE _____	**Country = 'USA'**	문자열 값 비교 시 작은따옴표('USA')와 등호(=)를 사용한다.
4	FirstName이 'Kim'인 직원 정보 조회 조건문: WHERE _____	**FirstName = 'Kim'**	FirstName 필드의 값이 'Kim'과 일치하는 행만 필터링한다.
5	FirstName만 오름차순 정렬: ORDER BY FirstName _____	**ASC** (또는 생략 가능)	ORDER BY는 정렬 구문이며 ASC는 오름차순(기본값)을 의미한다.
6	tracks에서 Milliseconds가 60,000보다 큰 데이터만 조회: WHERE _____	**Milliseconds > 60000;**	숫자 비교 연산자 >(초과)를 사용하며 SQL 숫자에는 쉼표를 쓰지 않는다.
7	중복을 제거하고 모든 국가 조회: SELECT _____ country FROM customers;	**DISTINCT**	DISTINCT는 SELECT 바로 뒤에 위치하여 중복을 제거하고 고유값만 반환한다.

번호	원본 문제 요지 및 빈칸/선지	정답	핵심 판단 근거 (해설)
8	고객 테이블에서 이름이 'son'으로 끝나는 사람 찾기: WHERE LastName _____ '%son'	**LIKE**	% 와일드카드를 사용한 문자열 패턴 매칭에는 =가 아니라 LIKE를 쓴다.
9	데이터베이스가 필요한 이유로 가장 적절한 것은?	**c) 데이터를 체계적으로 저장하고 활용하기 위해**	DB는 정보를 구조적으로 저장·검색·수정·삭제(CRUD)하기 위한 시스템이다.
10	관계형 데이터베이스에서 데이터를 저장하는 기본 단위는?	**b) 테이블**	RDB는 데이터를 행(Row)과 열(Column)로 구성된 테이블(Table) 단위로 저장한다.
11	기본 키(Primary Key)의 주요 목적은?	**c) 행을 고유하게 식별하기 위해**	테이블 내 각 행(레코드)을 중복 없이 유일하게 구별하는 식별자다.
12	외래 키(Foreign Key)는 언제 사용되는가?	**b) 두 테이블을 연결할 때**	다른 테이블의 기본 키(PK)를 참조하여 두 테이블 간 논리적 관계를 설정한다.
13	다음 중 RDBMS에 대한 설명으로 옳은 것은?	**c) 테이블 간 논리적인 관계를 지원한다**	관계형 구조를 지원하는 DBMS로서 여러 테이블을 키로 연결해 관리한다.
14	SQL은 어떤 목적으로 사용되는 언어인가?	**b) 데이터베이스 제어와 질의**	관계형 데이터베이스에서 데이터를 조회·조작·정의·제어하는 표준 질의 언어다.
15	SELECT 문은 주로 어떤 작업을 수행하는가?	**a) 데이터를 조회한다**	DQL에 속하며 테이블에서 원하는 데이터를 조회 및 반환한다.
16	WHERE 절의 역할로 알맞은 것은?	**d) 특정 조건의 데이터를 필터링한다**	지정한 검색 조건(비교/논리 연산)에 맞는 개별 행만 추출한다.

6.2 db 2.pdf (SQL 02) 확인 문제 1~16번 전수 해설 (원본 p.102~108)

번호	원본 문제 요지 및 쿼리	정답	핵심 판단 근거 (해설)
1	다음 중 테이블을 생성하는 SQL 문으로 올바른 것은?	**c) CREATE TABLE users (...);**	테이블 생성은 DDL인 CREATE TABLE을 사용한다(INSERT/SELECT TABLE 없음).
2	테이블에서 name과 age 컬럼만 선택적으로 조회하는 올바른 SQL 문은?	**c) SELECT name, age FROM users;**	기본 조회 구문은 SELECT 컬럼1, 컬럼2 FROM 테이블명;이다.
3	다음 중 UPDATE 문에 대한 설명으로 올바른 것은?	**b) 특정 데이터를 수정한다.**	DML인 UPDATE는 기존 레코드의 값을 수정한다(테이블 구조 수정은 ALTER TABLE).
4	외래 키(Foreign Key)의 기능으로 가장 적절한 것은?	**a) 다른 테이블과의 관계를 설정하는 기능**	부모 테이블의 기본 키를 참조하여 두 테이블 간의 관계를 연결한다.

번호	원본 문제 요지 및 쿼리	정답	핵심 판단 근거 (해설)
5	ALTER TABLE 명령어로 할 수 **없는** 작업은 무엇인가?	**d) 전체 테이블 삭제**	컬럼 추가·이름변경·삭제 및 테이블명 변경은 가능하나, 전체 테이블 삭제는 DROP TABLE로 수행한다.
6	오름차순으로 정렬된 결과를 얻기 위한 구문은?	**c) SELECT * FROM users ORDER BY age ASC;**	FROM 절 뒤에 ORDER BY 컬럼명 ASC를 붙여 오름차순 정렬한다.
7	DELETE FROM users WHERE id = 1; 명령어의 기능은?	**b) id가 1인 레코드만 삭제한다.**	WHERE id = 1 조건이 있으므로 조건에 맞는 특정 행만 삭제한다(전체 삭제는 WHERE 생략 시).
8	SELECT DISTINCT country FROM customers; 문장의 목적은?	**c) 중복 없이 고유한 국가명만 조회**	DISTINCT 키워드는 중복된 값을 제거하고 고유한 값만 반환한다.
9	기본 키(Primary Key)의 특징으로 가장 알맞은 것은?	**d) 각 레코드를 고유하게 식별한다.**	PK는 중복된 값과 NULL을 허용하지 않으며 각 행을 유일하게 식별한다.
10	SQL에서 LIKE '_a%' 조건의 의미는?	**a) 두 번째 글자가 'a'인 문자열**	_는 정확히 한 글자, %는 0개 이상의 문자이므로 두 번째 글자가 'a'인 패턴이다.
11	GROUP BY 절에 대한 설명으로 바른 것은?	**b) 특정 필드를 기준으로 데이터를 그룹화**	특정 열의 값이 같은 행들을 하나의 그룹으로 묶어 집계할 때 사용한다.
12	DROP TABLE employees; 명령이 실행되면 어떤 일이 발생하는가?	**b) employees 테이블이 삭제됨**	DROP TABLE은 데이터만 지우는 것이 아니라 테이블 전체(구조+데이터)를 삭제한다.
13	UPDATE customers SET Country = 'Korea' WHERE Country = 'KR'; 설명은?	**a) Country가 'KR'인 행만 'Korea'로 수정한다.**	WHERE Country = 'KR'로 대상을 좁힌 뒤 SET Country = 'Korea'로 값을 변경한다.
14	SELECT department, COUNT(*) AS cnt FROM employees GROUP BY department; 결과는?	**a) 각 부서의 직원 수를 구하여 부서 이름과 함께 보여준다.**	department별로 그룹을 묶고 각 그룹의 행 개수(COUNT(*))를 cnt 별칭으로 반환한다.
15	SELECT * FROM users WHERE name LIKE 'Lee%';는 어떤 레코드를 반환할까요?	**d) 이름이 'Lee'로 시작하는 사용자 모두 조회**	'Lee%'는 Lee로 시작하고 뒤에 0개 이상의 문자가 오는 조건이다(포함은 '%Lee%').
16	ALTER TABLE products ADD COLUMN stock INTEGER NOT NULL DEFAULT 0; 작업은?	**c) products 테이블에 stock 컬럼을 새로 추가하고 기본값을 0으로 지정한다.**	ADD COLUMN으로 정수형 stock 열을 추가하며 NOT NULL을 위해 기본값 0을 채운다.

6.3 db 3.pdf (Many To One Relationships 01) 확인 문제 1~14번 전수 해설 (원본 p.72~77)

번호	원본 문제 요지	정답	핵심 판단 근거 (해설)
1	related_name이 없을 때, Article 인스턴스	**b) article.comment_set.all()**	related_name 미설정 시 Django는

번호	원본 문제 요지	정답	핵심 판단 근거 (해설)
	article에서 올바른 역참조 코드는?		자동으로 '참조하는모델소문자명_set' 매니저를 생성한다.
2	id가 1인 게시글에 댓글을 생성하는 올바른 방법은?	**a) Comment.objects.create(...)**	ORM의 objects.create(article=article, content='새 댓글') 메서드로 생성한다.
3	[O/X] 'N'쪽 객체에서 '1'쪽 객체 필드 접근은 '역참조'이다.	**X (거짓)**	'N'쪽(외래키 보유)에서 '1'쪽으로 접근하는 것은 **'참조'**이고, '1'쪽에서 'N'쪽 목록에 접근하는 것이 **'역참조'**다.
4	Django 템플릿에서 댓글이 없는 경우를 처리하기 위해 사용하는 문법은?	**b) {% empty %}**	{% for comment in comments %} ... {% empty %} ... {% endfor %} 형태로 빈 목록 대체 콘텐츠를 출력한다.
5	admin.py에서 admin 사이트에 Comment 모델을 등록하는 함수는?	**admin.site.register** (admin.site.register(Comment))	admin.site.register(모델명)을 호출하여 관리자 페이지에 모델을 등록한다.
6	다음 중 N:1 관계를 올바르게 설명한 것은?	**b) 하나의 게시글에 여러 댓글이 달린다.**	여러 개의 하위 객체(N, 댓글)가 하나의 상위 객체(1, 게시글)를 참조하는 구조다.
7	댓글 모델에서 게시글을 참조할 때 사용하는 필드는?	**c) ForeignKey**	N:1 관계를 정의할 때는 models.ForeignKey 클래스를 사용한다.
8	[O/X] Django shell에서는 Comment.objects.create(article=article, content='내용')으로 댓글을 생성할 수 있다.	**O (참)**	create() 메서드에 참조할 객체(article)와 내용을 전달하여 즉시 DB에 생성할 수 있다.
9	[O/X] ForeignKey로 연결된 모델은 역참조 시 자동으로 related_name 속성이 생성된다.	**X (거짓)**	related_name은 수동 설정 옵션 이름이며, 미설정 시 자동 생성되는 것은 **소문자모델명_set(예: comment_set)**이다.
10	Django 댓글 생성 시 commit=False를 사용하는 이유는?	**b) 생성 후 필드 추가 설정**	DB 저장을 보류하고 인스턴스를 먼저 받아 외래 키(article, user 등)를 추가 설정한 뒤 저장하기 위함이다.
11	다음 중 N:1 관계에 해당하지 **않는** 예시는 무엇인가요?	**d) 여러 명의 배우가 여러 영화에 출연한다.**	배우도 여러 영화에 출연하고 영화도 여러 배우를 가지므로 이는 **N:M(다대다) 관계**다.

번호	원본 문제 요지	정답	핵심 판단 근거 (해설)
12	ForeignKey 필드 정의 시 필수 속성 2가지는? (주관식)	**참조할 모델 클래스(to / model class), on_delete**	참조 대상 모델과 부모 삭제 시 처리 방식(on_delete)은 필수로 작성해야 한다.
13	[O/X] 댓글 모델에서 게시글을 참조하도록 ForeignKey를 설정하면, 게시글에서 직접 참조 필드를 통해 댓글 목록을 조회할 수 있다.	**X (거짓)**	게시글(Article) 모델에는 직접 참조 필드가 없으므로, 역참조 매니저(article.comment_set.all())를 통해 거꾸로 조회해야 한다.
14	ForeignKey 필드명을 작성할 때 권장되는 방식은?	**c) 참조하는 모델의 클래스명을 소문자 단수형으로 쓴다.**	단일 객체를 가리키므로 article = models.ForeignKey(...)처럼 소문자 단수형 작성을 권장한다.

6.4 db 4.pdf` (Many To One Relationships 02) 확인 문제 1~16번 전수 해설 (원본 p.67~72)

번호	원본 문제 요지	정답	핵심 판단 근거 (해설)
1	Article이 User와 N:1 관계일 때 models.py의 외래 키 설정은?	**c) ForeignKey(settings.AUTH_USER_MODEL, ...)**	models.py에서는 User 모델을 안전하게 지연 참조하기 위해 settings.AUTH_USER_MODEL을 쓴다.
2	get_user_model() 함수는 주로 어디에서 사용하나요?	**c) models.py 외부 파일**	활성화된 User 모델 객체를 직접 반환하므로 views.py, forms.py 등 models.py 외부에서 쓴다.
3	기존 테이블에 외래 키 필드 추가(마이그레이션) 시 필요한 설정은?	**b) 기본값 입력**	기본적으로 NOT NULL 제약조건이 걸리므로 기존 행들에 채울 기본값(예: 1번 회원)을 입력해야 한다.
4	게시글 생성 시 작성자 정보를 저장하려면 어떤 작업이 필요한가?	**c) article.user = request.user**	form.save(commit=False) 이후 현재 로그인 사용자(request.user)를 직접 할당해 저장한다.
5	작성자 미지정 시 발생하는 에러는?	**b) NOT NULL constraint failed**	user_id 필드가 비어 있으면 DB의 NOT NULL 제약 조건 위반(IntegrityError)이 발생한다.
6	@login_required를 사용하는 이유는?	**c) 로그인한 사용자만 접근 허용**	인증된 사용자만 해당 뷰 함수를 실행할 수 있도록 사전 제한하는 데코레이터다.
7	require_POST의 기능은?	**b) POST만 허용**	해당 뷰가 오직 POST 요청만 처리하도록 제한하고 다른 요청에는 405 에러를 반환한다.

번호	원본 문제 요지	정답	핵심 판단 근거 (해설)
8	ERD에서 엔티티(Entity)는 무엇을 의미하나요?	**a) 테이블**	ERD에서 엔티티는 데이터베이스에 저장되는 개체로, 테이블(Table)로 표현된다.
9	ERD에서 속성(Attribute)은 무엇을 나타내나요?	**b) 컬럼 정보**	엔티티가 가지는 고유한 데이터 항목으로, 테이블의 컬럼(Field)으로 표현된다.
10	댓글 삭제 시 본인 확인을 위해 비교하는 값은?	**b) request.user와 comment.user**	삭제를 요청한 로그인 사용자(request.user)와 댓글 작성자(comment.user)가 같은지 비교한다.
11	템플릿에서 댓글 작성자와 내용을 함께 출력할 때 사용하는 코드는?	**c) {{ comment.user }} - {{ comment.content }}**	comment 객체의 참조 속성인 user와 본문 속성인 content를 나란히 출력한다.
12	수정·삭제 시 request.user와 비교하는 이유는?	**c) 작성자 본인인지 확인**	작성자 본인만 자신의 게시글이나 댓글을 수정·삭제할 수 있도록 인가(Authorization)하기 위함이다.
13	ERD를 작성하는 주된 목적은?	**b) 테이블 구조 시각화**	데이터베이스의 테이블(엔티티), 컬럼(속성), 관계(FK)를 시각적으로 모델링하여 구조를 파악하고 소통한다.
14	댓글 삭제 시 comment.user와 비교해야 할 로그인 유저 정보가 담긴 객체 ⓐ는?	**request.user**	Django에서 현재 HTTP 요청을 보낸 로그인 사용자 객체는 request.user에 담겨 있다.
15	settings.AUTH_USER_MODEL을 User 모델 직접 import 대신 사용하는 이유는?	**c) 에러 없이 안전하게 참조하려고**	앱/모델 로딩 순서에 상관없이 지연 평가(Lazy Evaluation)를 통해 안전하게 User를 참조하기 위함이다.
16	게시글 생성 시 작성자 정보를 저장하기 위해 article.user에 할당하는 값 ⓐ는?	**request.user**	로그인한 사용자의 정보를 게시글 작성자로 저장하기 위해 article.user = request.user를 수행한다.

Chapter 7. 평가 대비 7대 유형 심화 실전 문제 (판단 근거 상세 해설)

원본 62문항을 완벽히 소화한 학습자가 실제 코딩 테스트 및 서술·객관식 평가에서 만날 수 있는 **7대 핵심 유형(① 개념 설명, ② 원리 이해, ③ 조건 비교, ④ 실행 결과 예측, ⑤ 계산·코드 해석, ⑥ 오류 찾기, ⑦ 적용 상황 선택)** 심화 문제다.

1. **[유형 1: 개념 설명 — RDB 정규화와 무결성]** 게시글과 댓글을 하나의 board 테이블에 합쳐서 관리하지 않고, Article(1)과 Comment(N) 두 테이블로 분리한 뒤 Comment 테이블에 외래 키를 두는 이유를 '데이터 중복'과 '데이터 무결성' 관점에서 설명하시오.
2. **[유형 2: 원리 이해 — SQL 3값 논리와 집계 함수]** 총 10명의 직원이 있는 employees 테이블에서 ReportsTo 컬럼이 NULL인 직원이 2명, 1인 직원이 3명, 2인 직원이 5명이다. 다음 세 쿼리의 반환 결과를 각각 예측하고 그 원리를 설명하시오.

(A) SELECT COUNT(*) FROM employees WHERE ReportsTo = 1 OR ReportsTo != 1;
(B) SELECT COUNT(*), COUNT(ReportsTo) FROM employees;
(C) SELECT ReportsTo, COUNT(*) FROM employees GROUP BY ReportsTo ORDER BY ReportsTo ASC;의 첫 번째 행

3. **[유형 3: 조건 비교 — INNER JOIN vs LEFT JOIN 행 개수 변화]** users 테이블에 회원 3명(id: 1, 2, 3)이 있고, articles 테이블에 게시글 5개(userId: 1, 2, 1, 4, 1 — SQLite 외래키 미검사로 id=4 포함)가 있다. 다음 세 쿼리의 결과 행(Row) 개수는 각각 몇 개인가?
(A) SELECT * FROM articles INNER JOIN users ON users.id = articles.userId;
(B) SELECT * FROM articles LEFT JOIN users ON users.id = articles.userId;
(C) SELECT * FROM users LEFT JOIN articles ON articles.userId = users.id;

4. **[유형 4: 실행 결과 예측 — DQL 7단계 파이프라인]** 다음 쿼리를 실행했을 때 SQLite가 에러를 발생시키는 정확한 이유를 실행 순서 단계 번호를 들어 설명하고, 의도대로 동작하도록 수정하시오.

```
SELECT Country, COUNT(*) AS cust_cnt
FROM customers
WHERE Company IS NOT NULL AND cust_cnt >= 5
GROUP BY Country
ORDER BY cust_cnt DESC;
```

5. **[유형 5: 계산 및 코드 해석 — LIMIT/OFFSET과 서브쿼리 삭제]** 총 20개의 게시글(id 1~20, createdAt이 모두 다름)이 있는 articles 테이블에서, 최근 작성된 순서(내림차순)로 정렬했을 때 **11번째부터 15번째에 해당하는 게시글 5개를 한 번에 삭제**하는 SQL 문을 작성하시오.

6. **[유형 6: 오류 찾기 — Django 댓글 및 작성자 구현 버그 4곳 디버깅]** 다음 articles/views.py의 comments_create 코드에는 보안·런타임 에러·논리적 결함이 총 4곳 숨어 있다. 4곳을 모두 찾아 바르게 수정하시오.

```
@require_safe
def comments_create(request, pk):
    article = Article.objects.get(pk=pk)
    comment_form = CommentForm(request.POST)
    if comment_form.is_valid():
        comment = comment_form.save()
        comment.article_pk = article.pk
        comment.user = settings.AUTH_USER_MODEL
        comment.save()
        return redirect('articles:detail', article.pk)
```

7. **[유형 7: 적용 상황 선택 — 템플릿과 뷰의 이중 권한 제어]** 쇼핑몰 리뷰 게시판에서 작성자 본인만 리뷰를 삭제할 수 있도록 detail.html에 {% if request.user == review.user %}를 적용해 삭제 버튼을 숨겼다. 이로써 권한 제어가 완벽히 끝났는지 판단하고, 만약 부족하다면 어떤 공격 시나리오가 가능하며 어디에 어떤 코드를 추가해야 하는지 쓰시오.

심화 실전 문제 정답 및 판단 근거 해설

문항	정답 요약	판단 근거 및 핵심 논리 해설
1번	1쪽(Article)에 외래 키를 두면 댓글이 달릴 때마다 게시글 제목·내용이 중복 저장되어 공간 낭비와 수정 시 데이터 불일치(무결성 훼손)가 발생하므로, N쪽(Comment)에 article_id를 두어 1쪽의 PK만 참조하게 한다.	정규화의 핵심 목적은 데이터 중복 제거와 갱신 이상(Update Anomaly) 방지다. N쪽 테이블에 FK를 두면 게시글이 수정되어도 댓글 테이블은 전혀 건드릴 필요가 없어 참조 무결성이 유지된다(db 3.pdf p.14~15).
2번	(A) **8** (NULL인 2명은 제외됨!) (B) **10, 8** (C) **(NULL, 2)**	(A) ReportsTo가 NULL인 2명은 = 1도 UNKNOWN이고 != 1도 UNKNOWN이므로 둘 다 탈락하여 3 + 5 = 8명만 나온다! (B) COUNT(*)는 NULL 포함 전체 행(10), COUNT(컬럼)은 NULL 제외(8). (C) SQLite는 GROUP BY에서도 NULL을 하나의 그룹으로 묶고 ASC 정렬 시 맨 처음에 출력한다.

문항	정답 요약	판단 근거 및 핵심 논리 해설
3번	(A) **4행** (B) **5행** (C) **5행** (하석주 3행 + 송윤미 1행 + 유하선 1행)	(A) 4번 글(userId=4)이 탈락하여 1,2,3,5번 글 총 4행. (B) 왼쪽이 articles(5개)이므로 4번 글이 NULL과 함께 살아남아 5행. (C) 왼쪽이 users이므로 1번 하석주(글 3개 매칭 → 3행), 2번 송윤미(글 1개 매칭 → 1행), 3번 유하선(매칭 글 없음 → NULL로 1행 유지) 합계 **5행**이 반환된다!
4번	에러 원인: 2단계 WHERE에서 5단계 SELECT의 집계 별칭(cust_cnt)을 사용함. 수정: AND cust_cnt >= 5를 빼고 GROUP BY Country 뒤에 **HAVING COUNT(*) >= 5** 추가.	실행 순서가 ① FROM → ② WHERE → ③ GROUP BY → ④ HAVING → ⑤ SELECT → ⑥ ORDER BY이므로, 개별 행을 거르는 WHERE에는 Company IS NOT NULL만 남기고 그룹 집계 조건은 HAVING으로 옮겨야 한다.
5번	DELETE FROM articles WHERE id IN (SELECT id FROM articles ORDER BY createdAt DESC LIMIT 5 OFFSET 10);	"11번째부터 15번째"는 앞의 1~10번째(10개)를 건너뛰므로 OFFSET 10, 총 5개이므로 LIMIT 5다. 이를 서브쿼리로 감싸 WHERE id IN (...)에 전달한다.
6번	① @require_safe → **@require_POST** (+ @login_required) ② save() → **save(commit=False)** ③ comment.article_pk = ... → **comment.article = article** ④ settings.AUTH_USER_MODEL → **request.user**	① 댓글 생성은 DB 상태를 바꾸므로 @require_POST를 써야 한다. ② 외래 키 주입 전 즉시 저장하면 NOT NULL 에러가 난다. ③ article_pk라는 속성은 없으며 객체를 할당해야 한다. ④ settings.AUTH_USER_MODEL은 문자열('accounts.User')일 뿐 현재 로그인 유저 객체(request.user)가 아니다!
7번	불충분함! 공격자가 Postman이나 requests 모듈로 브라우저 UI를 우회해 직접 삭제 POST 요청을 보낼 수 있으므로, views.py의 삭제 함수 내부에도 반드시 **if request.user == review.user:** 검증을 추가해야 한다.	템플릿 조건문은 단지 화면(UI)에서 버튼만 안 보이게 할 뿐 서버 엔드포인트를 보호하지 못한다. 따라서 프론트엔드(템플릿)와 백엔드(views.py) 양쪽에서 이중 검증해야 안전하다 (db 4.pdf p.26).